

Individual Contribution Reflection

Name: Chong An

SID:550134350

Unikey:chan0893

1 Responsible Subsystem

In this assignment, I mainly implemented frontend UI and image-based question support, with partial authentication integration work.

--Image-based question variation (Major challenge)

Extended the existing question management flow to support an optional image URL field. This allowed admins to create image-based quiz questions without introducing file uploads. The image URL is stored with question data/available on the Internet and rendered conditionally on the player quiz page when available.

During the implementation of this function, major issues happened as these format:

***Integrating optional image-based questions into an existing text-only quiz flow while keeping backward compatibility with existing questions.**

This implementation allows three valid data states: a question with a valid image URL, a question with an empty imageUrl value, and an older question object that does not contain imageUrl at all. This was important because the variation should extend the existing system rather than force every previous question to be migrated.

--From frontend\src\pages\admin\QuestionForm.jsx, adding up an optional choice for ImageURL input.

```
const schema = z.object({
  text: z.string().min(1, "Question text is required"),
  imageUrl: z
    .string()
    .trim()
    .url("Image URL must be a valid URL")
    .optional()
    .or(z.literal("")),
  option0: z.string().min(1, "Option A is required"),
  option1: z.string().min(1, "Option B is required"),
  option2: z.string().min(1, "Option C is required"),
  option3: z.string().min(1, "Option D is required"),
  correctIndex: z.string().min(1, "Mark the correct answer"),
});
```

Note that this is the basic path for admins to upload quizzes. In the same file, a default value was

adjusted as None(“ ”) when left empty for sending them to backend database.

As a result, questions uploaded after this adjustment will grant a value of 'ImageUrl' with a real link/empty, but questions before this will have NO such attribute. This leads to a format fitting when getting questions from backend.

--From Quiz.jsx, I updated `Quiz.jsx` to conditionally render the image only when question.imageUrl exists. This prevents text-only questions from displaying broken image placeholders.

```
    {question.imageUrl && (
      <Box
        component="img"
        src={question.imageUrl}
        alt="Question image"
        sx={{
          width: "100%",
          maxHeight: 280,
          objectFit: "cover",
          display: "block",
        }}
        onError={(e) => {
          e.currentTarget.style.display = "none";
        }}
      />
    )}
```

This part is a rather simple solution for clashes on ImageURL attribute: Three cases could happen here: A true link, A link without getting anything(Same as “ ” in imageUrl attribute), No such value as 'ImageUrl' (As mentioned above). When two latter happen, we don't render anything. Only successful trials will get a render.

This solution improved robustness because the frontend no longer assumes that every question has an image. Instead, image rendering is treated as optional. If `imageUrl` is valid, the image is displayed; if it is empty, missing, or fails to load, the quiz continues as a normal text-based question. This kept the approved image-based variation compatible with the original core quiz mechanics.

--Admin login / admin authentication

Implemented the admin authentication flow, including checking the user role after login and restricting access to admin-only pages on the front end. Although this implementation was later refactored and replaced during team integration, it helped establish the expected behavior for separating normal users from admin users.

--Admin question management UI

Contributed to the frontend UI across the application, with a stronger focus on the normal user-side question management interface. This included improving page layout, form presentation, navigation flow, and consistency between the admin pages and the player-facing quiz pages.

--Frontend UI/UX polishing

Worked on UI/UX polishing, including layout consistency, clearer page structure, and better visual presentation for both normal text questions and image-based questions.

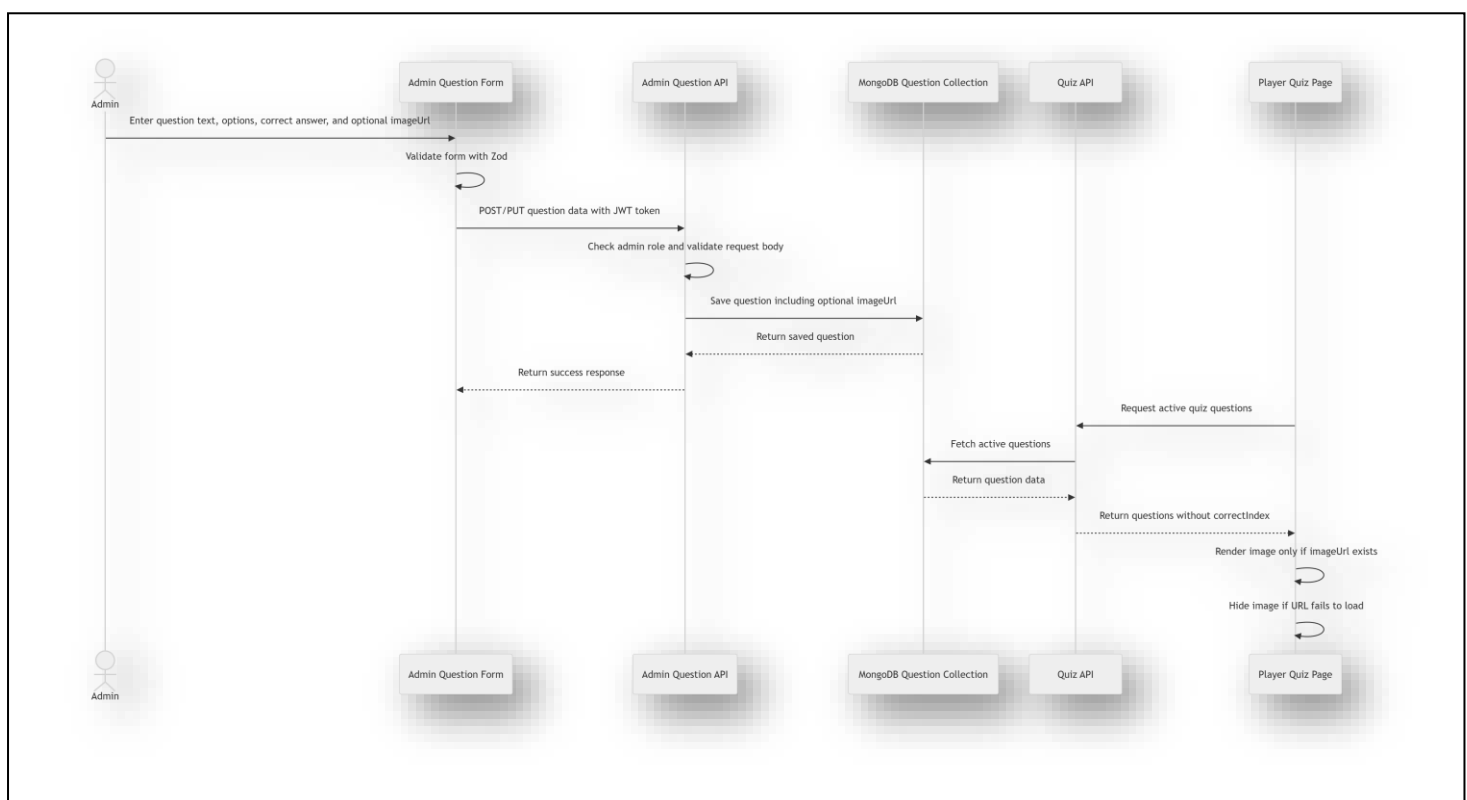
--Integration between frontend and backend

Worked on the integration between the frontend question forms and the backend question API. Ensured that the image URL entered from the admin interface could be sent to the backend, saved as part of the question object as an optional attribute, returned by the quiz API, and displayed correctly in the quiz interface.

--Validation / robustness / edge cases

Considered basic compatibility cases, such as allowing questions with/without ImageURLs to continue function, conditionally rendering images when an ImageURL exists in backend database, keeping the Image-based variation compatible with original quiz flow.

2 Mermaid sequence diagram



This sequence diagram describes my main subsystem: **the admin-side image-based question creation flow** and **the player-side rendering flow**.

On the admin side, the question form validates the input, including the optional imageUrl field, before

sending the data to the backend API. The backend then checks admin access, stores the question in MongoDB, and preserves the imageUrl field as part of the question data.

On the player side, the quiz page retrieves active questions from the quiz API. The correctIndex is not exposed to the player-facing response. When a question contains imageUrl, the frontend renders the image conditionally; otherwise, the question is displayed as a normal text-only multiple-choice question. This design keeps the approved image-based variation compatible with the original quiz mechanics.

3 Git Commit History

The following table analyses my directly attributable commits. Several of my commits were larger than ideal, I break them down into the concrete implementation units contained in each commit.

Commit hash	Implementation unit	Explanation
9f8d432	Admin identity support	Adjusted User.js to confirm that the system could represent an admin role
9f8d432	Basic admin route	Added an initial frontend route for the admin area
9f8d432	AdminPage prototype	Created an early AdminPage implementation. This page was later replaced during team integration.
9f8d432	AdminLogin prototype	Added an early AdminLogin page to separate admin login behaviour from normal user login.
9f8d432	Frontend centring/layout polish	Adjusted frontend page layout, form-based and central pages were visually aligned and placed consistently in the centre.
1639e05	Frontend typography polish	Continued polishing frontend typography and page presentation to make the UI more consistent.
1639e05	Local MongoDB login integration	Connected the login function with the local MongoDB environment so that authentication could be tested with real stored users rather than only frontend mock behaviour.
1639e05	Integration with remote changes	Merged remote changes while preserving the admin route work, helping align my frontend changes with the shared team codebase.

9ee0937	Auth controller fix	Modified auth.controller.js to improve the login/authentication flow.
9ee0937	Password hashing fix	Fixed the issue where login failed because user passwords were not being correctly encrypted or verified.
9ee0937	Authentication debugging	Debugged the current application flow and confirmed that the login process could run correctly after the fix.
d234492	Admin question form payload	Adjusted QuestionForm.jsx so that the frontend sends question data to the backend in the expected format, including the optional image URL field.
d234492	Admin image URL backend handling	Updated admin.controller.js so that image links could be accepted and passed through when creating or editing questions.
d234492	Quiz API image URL handling	Updated quiz.controller.js so that image URL data could be included in the player-facing question response while preserving the original quiz flow.
d234492	Quiz image rendering	Updated Quiz.jsx to read the adjusted question data format and render the image when imageUrl exists.
d234492	Home leaderboard component	Added HomesLeaderboard.jsx to enrich the opening page with extra leaderboard-related content.
d234492	Home page UI enhancement	Adjusted Home.jsx to include additional small UI components and improve the opening page presentation.

Note that Although the table above contains more than 12 implementation units, they are grouped under four larger commits.

I commit that this is a weakness in my development process. I should have committed smaller

changes more frequently, such as separating admin routing, authentication debugging, form payload changes, backend image URL handling, quiz rendering, and UI polishing into separate commits. Despite that, fewer commits have not caused any trouble in our group work.

4 Reflection on Design Decisions Related to the Approved Variation

Our group's approved variation was image-based questions. For this part, my main idea was to use an optional `imageUrl`` field first, instead of starting with file uploads. I chose this because it was a simpler and more realistic way to add images into the quiz system. The assignment was mainly about MERN structure, RESTful APIs, authentication, validation, and quiz logic, so adding a full upload system at the beginning would have made the feature more complicated than necessary.

I made `imageUrl`` optional because the original quiz system was still based on normal text questions. Some existing questions did not have any image field, and not every question needed to be image-based. If the image field was required, old questions or text-only questions might fail. By keeping it optional, both image questions and normal multiple-choice questions could work in the same quiz flow.

Another decision was to treat the image as part of the question display, not as a change to the quiz rules. The image only helps present the question. It does not change the number of options, the correct answer, answer submission, or scoring. This made the variation easier to add without changing the original quiz logic too much.

The main data flow was that the admin enters an image URL in `QuestionForm.jsx``, the backend receives it through `admin.controller.js``, the quiz API passes it through `quiz.controller.js``, and `Quiz.jsx`` renders the image if the field exists. This made the image-based variation work across both the admin side and the player side, instead of only being a frontend visual change. Later, another group member worked on an additional upload pathway, while my part focused on the core URL-based image question flow.

5 Integration, Validation, and Robustness

Our group had three members, so integration, validation, and robustness were not assigned to one single person in a very strict way. In practice, these responsibilities were shared across the group. My part was mainly related to the image-based question subsystem, early admin login/authentication work, and testing whether my changes worked correctly with the existing frontend and backend.

For the image-based variation, the main robustness issue I handled was compatibility with normal text-only questions. The original quiz system did not require images, so the frontend should not assume that every question has an `imageUrl`` field. To avoid breaking old or text-only questions, I made the image URL optional. If a question has a usable image URL, the image is displayed. If the value is empty, missing, or fails to load, the question still works as a normal multiple-choice question.

I also worked on keeping the image URL data flow consistent between the frontend and backend. The admin question form sends the optional `imageUrl`` as part of the question payload, the backend controllers pass it through as part of the question data, and the quiz page reads that field to render the image conditionally. This was important because the feature would not be complete if the image URL only existed on the frontend or only existed in the backend.

Besides the image feature, I also worked on early admin login and authentication-related testing. This included checking the admin identity flow, connecting login behaviour with the local MongoDB setup, and debugging password hashing/login issues. These tasks were not the final whole-system validation work, but they helped confirm that the admin-related pages and authentication flow could run properly during development.

Overall, my robustness work was focused on my own subsystem rather than the whole project. I mainly checked that image-based questions, text-only questions, admin question creation, and login-related behaviour did not break each other. This helped preserve the original quiz mechanics while adding the approved image-based variation.